

Full Stack Engineer Take-Home Assignment

Dear Candidate,

We would like to invite you to the next step in our evaluation process. Below is a technical task designed to assess your practical skills. Please read the instructions carefully.

Tech Stack: TypeScript, NestJS, PostgreSQL, Next

Time Limit

2 - 3 days from the time you receive this assignment.

Objective

Build a minimal **Multi-Tenant CRM System** that demonstrates:

- Backend architecture skills
- Database design knowledge
- Concurrency handling
- Performance awareness
- Clean TypeScript usage
- Frontend state management
- Production-level thinking

We value clarity, structure, and reasoning more than feature quantity.

Functional Requirements

1 Organizations

- An organization represents a company.
- Each user belongs to exactly one organization.
- Data must be fully isolated between organizations.

Users must never access another organization's data.

2 Users

Fields:

- id

- name
- email
- role (admin | member)
- organizationId

Rules:

- Only admins can create users.
- Users can only view data within their organization.

3 Customers

Fields:

- id
- name
- email
- phone
- organizationId
- assignedTo (User)
- createdAt
- updatedAt
- deletedAt (soft delete)

Requirements:

- Must support pagination
- Must support search (name/email)
- Must support soft delete
- Must not appear in normal queries if soft deleted

4 Notes

- Belong to a customer
- Belong to an organization
- Track createdBy

5 Activity Log

Log these events:

- Customer created
- Customer updated
- Customer deleted

- Customer restored
- Note added
- Customer assigned

Fields:

- entityType
- entityId
- action
- performedBy
- timestamp

Advanced Requirements (Mandatory)

1 Concurrency-Safe Assignment

Each user can have **maximum 5 active customers assigned**.

When assigning a customer:

- Must reject if user already has 5 active customers
- Must prevent race conditions
- Must work correctly under concurrent requests

Explain your approach clearly in README.

2 Performance Requirement

The system should support:

- 100,000 customers per organization

You must:

- Add appropriate database indexes
- Avoid N+1 queries
- Use efficient pagination

Explain performance decisions in README.

3 Soft Delete Integrity

When a customer is soft-deleted:

- They must not appear in normal queries
- Notes must remain stored
- Activity logs must remain stored
- Restoring customer must restore visibility of notes

4 Production Improvement

Implement **one production-grade improvement** of your choice.

Examples: (do not send examples)

- Rate limiting
- Logging middleware
- Caching
- Optimistic locking
- Request tracing
- Background job
- API documentation

Explain your choice and reasoning.



Frontend Requirements

Build a minimal UI with:

- Customer list
- Create/Edit customer
- Assign customer to user
- Add notes
- Pagination UI
- Loading states
- Error handling

Bonus:

- Debounced search
- Optimistic updates
- Reusable components



Technical Requirements

Backend:

- Use TypeScript strictly (no any)
- Use proper DTO validation
- Clean folder structure
- Separate controller/service layers
- Use transactions where required

- Proper foreign keys
- At least one manual index

Frontend:

- Clean state management
- Proper error handling
- Avoid unnecessary re-renders
- Type-safe API calls



README Requirements

Include:

1. Architecture decisions
2. How multi-tenancy isolation is enforced
3. How concurrency safety is achieved
4. Performance strategy and indexing decisions
5. How you would scale this system
6. Trade-offs made
7. Production improvement explanation



Submission

- Provide GitHub repository link
- Provide deployed url
- Include setup instructions
- Include seed data (optional but appreciated)

We care about:

- Code clarity
- Data consistency
- Thought process
- Structure
- Reasoning ability

We do not care about:

Fancy UI

- Overengineering
- Perfect styling